

Project 2: Online Retail Forecasting — Team Evaluation & Improvement Plan

1. SUMMARY OF CURRENT STATUS

Data Pipeline

- **Source:** UCI Online Retail II (1.07M transaction rows, Dec 2009–Dec 2011)
- **Processing:** UK-only filter (~92% of data), LLM-based (GPT) product family classification from raw descriptions, stock-code imputation for missing descriptions
- **Target:** $\text{total_sales} = \text{Quantity} \times \text{Price}$, aggregated to **daily** level per product family
- **Split:** Chronological — train $\leq$ 2011-06-08, val 2011-03-15 to 2011-06-08 (last 15% of training days), test $>$ 2011-06-08. Matches prior team exactly for apples-to-apples MAPE comparison.

Clustering

- **Method:** K-Means (K=5), chosen over GMM and HDBSCAN
- **Feature set:** 50 time-series features (intermittency, ACF, STL, volatility, trend, Croston/TSB summaries, business context)
- **Result:** 6 clusters after C1 split — C0 Erratic (n=515), C1a Seasonal (n=317), C1b Non-Seasonal (n=156), C2 Dense-HiVol (n=10), C3 Sparse Long-Tail (n=1017), C4 Ultra-Sparse (n=8)

Models (as of Phase 2 completion)

Cluster	Prior Team Baseline	Prior Team Model	Our Model
C0 (Erratic)	TSB	Two-Stage LightGBM	LGBM (retrained on corrected daily data)
C1a (Seasonal-Interrmittent, $\text{STL} \geq 0.5$)	Naive 7-Day	ZINB (all C1)	Prophet
C1b (Non-Seasonal, $\text{STL} < 0.5$)	Naive 7-Day	ZINB (all C1)	LGBM
C2 (Dense HiVol)	Holt-Winters/ETS	LightGBM	Prophet + Global LGBM
C3 (Sparse Long-Tail)	Naive 7-Day	Two-Stage HistGradientBoosting	DeepAR
C4 (Ultra-Sparse)	Not modeled	Manual/rule-based	SWLY rule

Prior Team's Reported MAPE Scores (baseline to beat)

Cluster	Model	WMAPE	ϵ -MAPE	Non-Zero MAPE
C0	TS-LGBM	94.89%	340.67%	155.94%
C1	ZINB	126.09%	173.05%	160.76%
C2	G-LGBM	76.06%	339.31%	96.79%
C3	TS-HGB	150.11%	112.20%	154.44%

2. WHAT WAS DONE IN PHASES 1 & 2 (COMPLETED)

Phase 1 — Rebuilt Daily Data Pipeline (notebook 02)

Three data quality issues fixed versus the prior team:

1. Net-negative daily sales clipped to 0. The prior team filtered `Quantity < 0` at the transaction row level, but returns and sales for the same product family on the same day aggregate together — a family with more returns than purchases still produced a negative `total_sales`. We clip to 0 after aggregation. This affected 14,525 day-product pairs across 1,088 products.

2. Winsorization at 99th percentile per product. 886 products have $p_{99} > 10\times$ their median daily sales — mostly Christmas spikes. These dominate training loss and cause systematic underprediction on normal days. Caps are fit on train only and saved to `winsor_caps.parquet`. Raw (unwinsorized) splits are kept separately for MAPE evaluation against true values.

3. Holiday and calendar features added. The prior team had no holiday encoding. We added:

- `days_to_holiday` — signed distance (± 30 cap) to the nearest UK bank holiday. UK bank holidays have zero transactions in this dataset (shops close), so the holiday day itself doesn't appear in the daily grid. `is_holiday` was dropped for this reason — it is always 0. The days *around* holidays drive demand surges.
- `is_christmas_week` — 1 if ISO week 51 or 52.
- Day-of-week, week-of-year, month cyclical encodings (sin/cos) to prevent ordinal leakage at year boundaries.
- 28-day rolling unit price per product — captures promotional discounting signals.

Log-transform: Not applied at the pipeline level. This is a model-level decision to be made inside each model's notebook (e.g., Prophet handles it natively via `log` transform option; LGBM can use log-transformed target).

Output files:

File	Description
<code>data/daily_train_clustered.parquet</code>	Unwinsorized train — use for evaluation
<code>data/daily_train_clustered_winsorized.parquet</code>	Winsorized train — use for model training
<code>data/winsor_caps.parquet</code>	Per-product p99 caps, fit on train only

Phase 2 — Clustering Improvements (notebooks 03 & 04)

C1 split (notebook 03). The prior team routed all 473 C1 products to Prophet. But 156 have `stl_seasonality_strength < 0.5` on the daily series (period=7, day-of-week), meaning no meaningful day-of-week seasonality. Applying Prophet to non-seasonal series inflates MAPE. We split:

- **C1a** (317 products, daily STL ≥ 0.5) → Prophet
- **C1b** (156 products, daily STL < 0.5) → LGBM (same treatment as C0)

We used daily STL (period=7) rather than weekly STL (period=52) because: (a) C1's defining seasonality is day-of-week, not annual; (b) weekly STL is unreliable with only ~73 training weeks (1.4 annual cycles).

Re-clustering experiment (notebook 04). The prior team's 50-feature set has 7 pairs with $r \geq 0.999$ (exact duplicates like `nonzero_rate ~ tsb_p`, `adi ~ croston_mean_interval`). K-Means double-weights correlated dimensions, biasing cluster geometry. We re-ran K-Means after dropping 7 redundant features (43-feature clean set):

- **ARI = 0.54** against original assignments — substantial reshuffling, not stable
- **Silhouette dropped by 0.05** — geometry got worse, not better

Decision: keep original assignments. The redundant features are all intermittency metrics that are genuinely the strongest clustering signal in this dataset. Removing them hurts more than the double-weighting.

3. GAPS & HOW TO CLOSE THEM

Gap 1 — No probabilistic / deep-learning models (professor explicitly called this out)

Problem: Prior team used only classical (ETS, TSB, ZINB) or tree-based (LGBM, HGB) models. No TFT, DeepAR, or Prophet. Required by professor feedback.

Fix:

- **Prophet** on C1a (seasonal-intermittent) and C2 (dense). Prophet natively models weekly + annual seasonality, holiday effects, and trend changepoints.
- **DeepAR** for C3 (sparse long-tail) via GluonTS. DeepAR's negative binomial head pools across all 1017 C3 SKUs.
- **TFT** (optional, if time allows) as a global model for C0 and/or C2. Use PyTorch Forecasting library.
- **LGBM** retained for C0 and C1b as a strong tree-based model.

Literature evidence:

Paper	Model	Key Result
Lim et al. 2019 (arxiv 1912.09363)	TFT	P50 loss 0.354 vs DeepAR 0.574 on retail
Salinas et al. 2020 (IJF)	DeepAR	Purpose-built for sparse count-like retail series
Zhang et al. 2023 (Springer AOR)	Transformer	Outperforms Croston on sparse intermittent demand

Paper	Model	Key Result
Hobor et al. 2025 (arxiv 2506.05941)	LGBM vs TFT	Tree-based wins on ultra-sparse; neural wins on moderate-density

Gap 2 — MAPE scores are very high (all >76%)

Problem: Systematic underprediction from outlier spikes + no holiday effects.

Fix (already done in Phase 1):

- UK holiday encoding via `days_to_holiday` and `is_christmas_week` ✓
- Winsorization at 99th percentile per product, fit on train only ✓
- Log-transform is a model-level decision to be applied inside Prophet/LGBM notebooks

Gap 3 — Forecasting granularity decision: daily for models, weekly for agent

Decision: Train and evaluate all models at **daily** granularity. The agent outputs 12-week forecasts by aggregating daily predictions.

Rationale:

- C1's defining seasonality is day-of-week (STL period=7). Aggregating to weekly collapses this pattern entirely.
- Daily training gives 511 data points vs 73 weeks — far more reliable model fitting.
- Keeps evaluation apples-to-apples with the prior team (same test set, same granularity).
- Prophet period=7 is coherent with daily data.
- The agent's business-facing output can still be weekly by summing 7-day windows after prediction.

Weekly pipeline (notebook 01) is kept only for the Streamlit agent's data layer — it produces weekly aggregates for display purposes, not for model training.

Gap 4 — Feature engineering (partially resolved)

Added in Phase 1 (pipeline-level, shared across all models): UK bank holidays (`days_to_holiday`, `is_christmas_week`), cyclical encodings (sin/cos for DOW, week, month), price trend (28-day rolling).

✓

Still to add inside LGBM model notebooks (not needed in pipeline; model-specific):

- **EWM features:** `ewm_mean_7`, `ewm_mean_28`, `ewm_sale_prob_14` — exponential weighted moving averages that smooth intermittency better than rolling means; prior team used these in C0/C2.
- **Days-since-last-sale:** clipped to 56 days — strong signal for intermittent demand occurrence prediction.
- **Sale-rate windows:** 7-day and 28-day rolling proportion of days with nonzero sales.
- **SKU profile stats** (train-derived, static): mean, std, nonzero rate, ADI, CV² per product — provides demand regime context to the global model.

Gap 5 — C4 cluster is abandoned

Fix: Apply a **same-week-last-year (SWLY) naive forecast** for C4 (8 products). Since STL=1.0 (pure seasonal), last year's value in the same calendar week is the best forecast. ~10 lines of code; satisfies rubric "optimization" requirement.

Gap 6 — No interpretability / SHAP analysis

Fix:

- **SHAP waterfall plots** for C0 and C1b LGBM models on representative SKUs.
- **Prophet `plot_components()`** for C1a and C2 — free interpretability.

Gap 7 — Evaluation metric alignment with rubric

Fix:

- Report **standard daily MAPE** as the headline metric alongside WMAPE.
- Single clean comparison table: [Prior team model, Our model] × [Cluster, MAPE%] at daily level.

Gap 8 — No prediction intervals (prior team reported only point forecasts)

Fix: Add 80%/95% prediction intervals for all models. Report empirical coverage on the test set. Prior team had none — this is an easy win for the report's statistical contribution section.

Gap 9 — No hyperparameter optimization

Fix: Systematic Optuna search (20–50 trials) for Prophet and LGBM. Prior team used defaults or hand-tuned. Typically yields 5–10% MAPE improvement and is easy to justify under the rubric's "optimization" criterion.

4. HOW PROFESSOR'S COMMENTS WERE INCORPORATED

Professor Comment	Our Response
"Use TFT or DeepAR, AND Prophet"	DeepAR for C3, Prophet for C1a/C2, LGBM for C0/C1b; TFT optional if time allows
"Their MAPEs are high (not penalizing)"	Address via holiday features + winsorization + better model routing
"You can solve an entirely different problem (hourly vs daily)"	We stay at daily granularity for models (best for C1's day-of-week seasonality); agent output is weekly
"Leverage their code but not required, clean slate ok"	We reuse their data pipeline outputs and cluster assignments; rebuilt data pipeline and clustering from scratch
"Applying techniques from papers presented in class"	DeepAR (Salinas et al. 2020) and Prophet are standard course-covered methods

5. RUBRIC COVERAGE

PRESENTATION

Criterion	How We Cover It
Executive Summary & Value Creation	Lead with: "We reduced daily MAPE from X% to Y% using Prophet+DeepAR, enabling better inventory planning"
Identification of Improvements	Explicit table: holiday features, C1 split, model upgrades, interpretability
MAPE Comparison Table	Single dedicated slide: prior team MAPE vs our MAPE by cluster at daily level
Next Steps	C4 SWLY rule, TFT, real-time data feeds, ensemble stacking

TECHNICAL REPORT

Criterion	How We Cover It
Architecture & Modeling Changes	Document the daily pipeline, C1 split, DeepAR/Prophet additions, justify each choice
Feature Engineering & Rationale	Document holiday addition (days_to_holiday, is_christmas_week), cyclical encoding, price trend, winsorization
Future Work	TFT hyperparameter tuning, online learning, deployment

CODE

Criterion	How We Cover It
Data Splitting Strategy	Strict chronological split: train $\leq$ 2011-06-08, val 2011-03-15 to 2011-06-08, test $>$ 2011-06-08
Evaluation Metric (MAPE)	Standard daily MAPE as primary, WMAPE as secondary
Optimization Techniques	Holiday features, winsorization, C1 split, deep learning models
Software Engineering	Clean modular notebooks: <code>data/</code> , <code>notebooks/</code> , <code>agent/</code> ; descriptive names

6. IMPLEMENTATION SEQUENCE

Phase 1 — Daily Data Pipeline COMPLETE

- Rebuilt from raw `_post` parquets at daily granularity
- Fixed net-negative clipping, added holiday features, winsorized
- Output: `daily_train/val/test_clustered.parquet` (raw + winsorized)

Phase 2 — Clustering Improvements COMPLETE

- C1 split: C1a (STL $\geq$ 0.5, n=317) $\rightarrow$ Prophet; C1b (STL $<$ 0.5, n=156) $\rightarrow$ LGBM
- Re-clustering experiment: kept original (ARI=0.54, silhouette -0.05)

- Output: `data/clustering/clusters_final.parquet`

Phase 3 — Baseline Models (next)

Per-cluster baselines to include in the comparison table:

Cluster	Baselines
C0	TSB ($\alpha=0.15$, $\alpha p=0.10$), Naive-7
C1a	Naive-7, iMAPA (aggregation levels 7/14/28)
C1b	Naive-7, iMAPA
C2	Holt-Winters/ETS, SMA-28
C3	Naive-7, TSB, iMAPA, ZINB (prior team's best for this cluster)
C4	SWLY (same-week-last-year)

iMAPA is an ensemble of ADIDA at multiple aggregation scales — the prior team found it competitive with ZINB on C3. Including it sets a harder bar for DeepAR to beat.

Phase 4 — Primary Models

C0 & C1b — Two-Stage LGBM

Reusing from prior team (proven architecture):

- Two-stage design: Stage 1 `LGBMClassifier` for occurrence (sale/no-sale), Stage 2 `LGBMRegressor` on positive-sales days only
- Recency-based sample weights (recent 28 days upweighted, max 3x)
- Tuning loss: $0.6 \times \text{WMAPE} + 0.4 \times \text{NonZeroMAPE}$
- Feature types: lags (1, 7, 14, 28), rolling mean/std (7, 28), EWM (span 7/28), days-since-last-sale (clip 56 days), sale-rate windows

Our additions:

- Retrained on corrected daily data (net-negative clipped, winsorized)
- Holiday features added as regressors: `days_to_holiday`, `is_christmas_week`
- C1b products newly routed here (prior team used ZINB for all C1)
- Systematic Optuna tuning (50 trials) for `num_leaves`, `learning_rate`, `min_child_samples`, threshold τ , and α scaling — prior team used hand-tuned grids
- SHAP waterfall plots (prior team had no interpretability)

C1a & C2 — Prophet (*new*; prior team used ZINB / LGBM):

- Extra regressors: `days_to_holiday`, `is_christmas_week`
- `log` mode for skewed series; `uncertainty_samples=200` for prediction intervals
- Tune `changepoint_prior_scale` and `seasonality_prior_scale` via Optuna (30 trials)
- `plot_components()` for free interpretability

C3 — DeepAR (*new*; prior team used Two-Stage HGB):

- GluonTS, negative binomial output head, global model pooling all 1017 C3 SKUs
- Static covariates: `sku_code`; time-varying: all calendar + holiday features from pipeline
- Fallback: if DeepAR val MAPE > iMAPA baseline, use ZINB (prior team showed ZINB is competitive on C3)

C4 — SWLY rule (*formalized; prior team left C4 unmodeled*): same calendar-week value from prior year, ~10 lines

TFT (optional, time permitting) — global model for C0/C1a; `hidden_size=32`, `attention_head_size=2`, ≤50 epochs on CPU; `cluster_id` as static covariate. Satisfies "deep learning" rubric requirement even if Prophet wins on val.

Ensemble (optional) — per-cluster Ridge-weighted average of all individual forecasts, weights fit on val set. Typically yields 5–15% MAPE reduction.

Phase 5 — Evaluation

- Daily MAPE, WMAPE comparison table vs prior team by cluster
- SHAP for LGBM, Prophet component plots, DeepAR probabilistic intervals
- **Prediction intervals:** Prophet (`uncertainty_samples`), DeepAR (sample from NegBinomial head), LGBM (quantile objective at $q=0.1/0.9$). Report empirical coverage ("X% of actuals fell within our 80% interval") — a contribution the prior team missed entirely.

Phase 6 — Agent (update routing)

- Update Streamlit agent routing to reflect C1a/C1b split and new model artifacts
- Agent output: sum daily predictions to weekly windows for 12-week display

7. EXPERT NOTES / ADDITIONAL CONSIDERATIONS

1. **Daily MAPE expectation:** At daily granularity with holidays encoded, expect improvements over prior team but numbers will remain high (daily MAPE is inherently higher than weekly). Focus comparison on % improvement over prior team's daily baseline, not absolute level.
2. **TFT training time:** TFT requires GPU for reasonable training time. If running on CPU only, reduce to 50 epochs and use `hidden_size=32`. DeepAR via GluonTS is more forgiving on CPU.
3. **DeepAR on GluonTS:** For C3 (1017 sparse SKUs), pooling them into one DeepAR model is exactly the intended use case. Use negative binomial output distribution for count-like demand.
4. **Prophet limitations:** Prophet does not handle zero-inflated series well. Use it for C2 (dense) and C1a (seasonal, moderate zeros) only. C1b and C0 go to LGBM.
5. **Do NOT rerun LLM classification:** The `desc2fam_checkpoint.jsonl` and post-processed parquets exist. Rerunning costs API money and adds no value.
6. **Cluster C4 is only 8 products:** A SWLY rule takes ~10 lines of code. It satisfies rubric "optimization" requirement at near-zero cost.

7. **Presentation strategy:** Lead with the MAPE improvement number on slide 1. Use a simple bar chart: "Prior team: X% → Our team: Y%" by cluster.
 8. **Report strategy:** Summarize prior team's methodology in 1-2 pages, then dedicate the rest to our additions. Cite CHANGES.md as a source of what we changed and why.
 9. **Code quality for rubric:** Use descriptive function names, module structure, Python logging (not print), docstrings on public functions, no bare `except:` clauses.
 10. **Key library versions:** PyTorch Forecasting needs PyTorch ≥ 1.9 ; GluonTS works with MXNet or PyTorch backend. Check compatibility with existing `environment.yml` before committing.
-

8. ARXIV LITERATURE CITATIONS FOR REPORT

1. **TFT:** Lim et al. (2019). *Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting*. arxiv:1912.09363.
2. **DeepAR:** Salinas et al. (2020). *DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks*. IJF.
3. **Switch-Hurdle:** (2026). *Switch-Hurdle: A MoE Encoder with AR Hurdle Decoder for Intermittent Demand Forecasting*. arxiv:2602.22685.
4. **Transformer for Intermittent Demand:** Zhang, Xia, Xie (2023). *Intermittent demand forecasting with transformer neural networks*. Annals of Operations Research.
5. **N-HiTS:** Challu et al. (2023). *N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting*. arxiv:2201.12886.
6. **PatchTST:** Nie et al. (2023). *A Time Series is Worth 64 Words*. ICLR 2023. arxiv:2211.14730.
7. **Hobor et al. (2025).** *Comparative Analysis of Modern ML Models for Retail Sales Forecasting*. arxiv:2506.05941.